Defining a Class:

Is defined as a **user-defined dataType** which includes **Fields** (Variables or **Data Members**) and **Methods** (**Functions** OR **Member Functions**) in a **single unit**.

Syntax:

```
class class_name
{
        [variable(s) declaration];
        [methods declaration];
}
```

Adding Variables:

Data is **encapsulated** in a class by placing **data fields** inside the **body** of the **class** definition. These variables are called as **Instance variables**.

Syntax:

```
class Rectangle
{
        int length;
        int width;
}
```

Adding Methods:

Data is **encapsulated** in a class by placing **data fields** inside the **body** of the **class** definition. These variables are called as **Instance variables**; also we can place **methods** inside the class body as follows.

Syntax:

```
return_type method_name (parameter-list)
{
        Method-body;
}
```

```
class Rectangle
{
        int length;
        int width;
        void getdata(int x, int y)
        {
                Length =      x;
                Width  =      y;
        }
}
```

Creating Objects:

An **Object** is defined as an **Instance** of **class** and also it can consider as a **basic run-time entity**. As pointed out earlier that, an **object** in Java is **essentially** a **block of memory** that contains space to **store** all the **instance variables**. Creating an object is also **referred** to as **instantiating** an **Object**.

Objects in Java are **created** using the **new operator**. The new operator **creates** an **object** of the specified class and returns **reference** to that **object**.

Syntax:

```
class_name object_name;
object_name = new class_name();
          OR
class_name object_name = new class_name();

Example:    Student S = new Student();
```

Accessing Class Members:

All the **variables** must be **assigned values before** they are **used**. Since we are **outside** the **class**, we **cannot access** the **instance variables** and **methods directly**. To do this, we must use the concerned **object** and the **dot (.)** operator as shown below.

Syntax:

```
object_name. variable_Name;

object_name. method_Name (parameter-list);
```

```
class Rectangle
{
     int length, width;
     int getdata(int x int y)
     {
      length = x;
      width = y;
     }
     int rect_area()
     {
      int area = length * width;
      return(area);
     }
}
class Rect
{
     public static void main(String args[])
     {
```

```
        int area1, area2;
        Rectangle R1 = new Rectangle();
        Rectangle R2 = new Rectangle();
        R1.length = 15;
        R1.width = 10;
        area1 = R1.length * R1.width;
        R2 = R2.getdata(25, 30);
        area2 = R2.rect_area();
        System.out.println(" The Area of Rectangle using object R1 =" +area1);
        System.out.println(" The Area of Rectangle using object R2 =" +area2);
      }
}
```

Output:

```
The Area of Rectangle using object R1 = 150

The Area of Rectangle using object R2 = 750
```

Constructor in Java:

A **constructor** in Java is a **special member function** that is **called** when an **object** is **created**. It **initializes** the class objects. It is used to **set default** or **user-defined values** for the class objects.

- **Constructor** has the **same name** as the **class name**.
- It does not have a **return type**, not **even void**.
- It can **accept parameters** to **initialize** object properties.

Types of constructors:

1. Default Constructor
2. Parameterized Constructor
3. Copy Constructor

1. **Default Constructor:** A **default constructor** has **no parameters**. It's used to **assign default values** to an **object**. If no constructor is **explicitly defined**, Java provides a **default constructor**.

2. **Parameterized Constructor:** A **constructor** that **has parameters** is known as **parameterized constructor**. If we want to **initialize fields** of the class with our **own values**, then use a **parameterized constructor**.

3. **Copy Constructor:** A **constructor** which **initializes the object** using another **object**. In this type of constructor parameters are of object type.

Program to demonstrate the use of constructor, types of constructor, overloaded constructor.

```
import java.lang.*;
class Student
{

  String Name;
  int Regno;
  int Marks1, Marks2, Marks3;
  Student ()                    // Default Constructor
   {
     Name="raju";
     Regno=12345;
     Marks1=56;
     Marks2=47;
     Marks3=78;
   }
  Student (String N, int r, int m1,int m2,int m3)
   {
     Name=n;
     Regno=r;
     Marks1=m1;
     Marks2=m2;
     Marks3=m3;
   }
    Student (Student S)
   {

     Name=s.name;
     Regno=s.regno;
     Marks1=s.marks1;
     Marks2=s.marks2;
     Marks3=s.marks3;
   }

  void display()
   {
     System.out.println(name+"\t"+regno+"\t"+marks1+"\t"+marks2+"\t"+marks3);
   }
}
class StudentDemo
{

  public static void main(String args[])
   {

    Student S1=new Student();
    Student S2=new Student("Akshara",12345,75,89,96);
    Student S3=new Student(S1);
    S1.display();
    S2.display();
    S3.display();
   }
}
```

Method Overloading in Java:

Method Overloading in Java allows a class to **have multiple methods** with the **same name** but **different parameters**, enabling compile-time **polymorphism**.

- Methods can share the **same name** if their **parameter lists differ**.
- Cannot **overload** by **return type** alone; **parameters must differ**.

Example-1:

```
import java.io.*;
class Product
{
	public int multiply(int a, int b)
	{
		int prod = a * b;
		return prod;
	}
	public int multiply(int a, int b, int c)
	{
	 int prod = a * b * c;
	 return prod;
	}
}
class Sample
{
	public static void main(String[] args)
	{
	 Product P = new Product();
	 int prod1 = P.multiply(1, 2);
	 System.out.println ("Product of the two integer value: " + prod1);
	 int prod2 = P.multiply(1, 2, 3);
	 System.out.println ("Product of the three integer value: " + prod2);
	}
}
```

Example-2:

```
class Product
{
	public int prod(int a, int b, int c)
	 {
		return a * b * c;
	 }
	public double prod(double a, double b, double c)
	 {
		return a * b * c;
	 }
}
public class Sample
 {
	public static void main(String[] args)
	{
		Product p = new Product();
		System.out.println(p.prod(1, 2, 3));
		System.out.println(p.prod(1.0, 2.0, 3.0));
	}
 }
```

Method Overriding in Java:

The **method defined** in the **Base Class** is **Inherited** by its **subclass** and **used** by the **objects** created by the **subclass**. Method Inheritance enables us to **define** and **use methods repeatedly** in subclasses without having to define the methods **again** in the **subclass**.

However, there may be some occasions when we want an **object** to **respond** to the **same methods** but **have different behavior** when that **method** is **called**. That means, we should **override** the **method defined** in the **Base Class**. This is possible by **defining** a **method** in the **subclass** that has the **same name**, **same arguments** and **same return type** as a method in the **Base Class**. Then, when that method is called, the **method** defined in the **subclass** is **invoked** and **executed instead** of the one in the **Base Class**. This is known as **Method Overriding**.

```
import java.io.*;
class Base
{
     int x;
     Base(int x)
     {
      This.x = x;
     }
     void display()
     {
      System.out.println("Base Class x = " + x);
     }
}

class Sub
{
     int y;
     Sub(int x, int y)
     {
       Base(x);
      This.x = x;
     }
     void display()
     {
      System.out.println("Base Class x = " + x);
      System.out.println("Sub Class y = " + y);
     }
}
class Override
{
   public static void main(String[] args)
     {
      Sub S = new Sub(100, 200);
      S.display();
     }
}
```

Output:

```
Base Class x = 100
Sub Class y = 200
```

Final Variables in Java:

In Java, a **`final` variable** is a variable whose value can be assigned **only once**. Once **initialized**, it **cannot** be **changed**.

```
final DataType Variable_Name = value;
```

```
public class Sample
 {
    public static void main(String[] args)
     {
        final int AGE = 25;

        System.out.println(AGE);

       AGE = 30;          // Error: cannot assign a value to final variable AGE
    }
}
```

- final variable can be declared without initialization, but it must be initialized exactly once before it is used.

```
public class Student
{
    final int rollNo;

    Student(int rollNo)
    {
        this.rollNo = rollNo;
    }

    public static void main(String[] args)
    {
        Student S = new Student(101);
        System.out.println (S.rollNo);
    }
}
```

Key Points

- **final variables** can be **initialized**:
  1. At the time of declaration.
  2. Inside a constructor (for instance variables).
  3. Inside a static block (for static final variables).
- final primitive variable's value **cannot change after initialization**.
- final reference variable **cannot refer** to a different object, but the object's state can still change.
- By convention, static final constants are written in uppercase:
- final is commonly used to create **constants**, **prevent** accidental **modification**, and **improve code readability** and **safety**.

Final Methods in Java:

final method in Java is a method that **cannot be overridden by a subclass**. The **final keyword** is used when you want to **prevent** a **method's implementation** from **being changed** through **method overriding**.

```
class Animal
{
    final void sound()
    {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal
{
    void sound()
      {
         System.out.println ("Dog barks");
      }
}

public class Sample
{
    public static void main(String[] args)
      {
        Dog d = new Dog();
        d.sound();
      }
}
```

Note: The above program **do not let us** to **redefine** the **base** class **final method again** in the **subclass**. Here, we need to change the method name sound () in the subclass. The compiler gives an error because a **final method cannot be overridden**.

Why use final methods?

Final methods are used to:

- **Prevent** method **overriding**
- **Protect important** or **critical** implementation
- Maintain **consistent behavior** in **subclasses**
- Improve **program security** and **reliability**

Important Points

- final method **can be inherited** by a subclass.
- final method **cannot be overridden**.
- final method can be **called normally** from a subclass.
- **Constructors** cannot be **declared final**.
- **private methods cannot** be **overridden**, so declaring them final is **generally unnecessary**.

Final Classes in Java:

final class in Java is a class that **cannot be inherited (extended) by another class**. The **final keyword** is **used with** a **class** when you want to **prevent inheritance**.

```
final class Animal
{
    void sound () {
        System.out.println ("Animal makes a sound");
    }
}

// This is not allowed

class Dog extends Animal
{
    // Error: cannot inherit from final Animal
}
```

```
final class Student
{
    void display ()
     {
        System.out.println ("Student class");
     }
}

public class Main
{
    public static void main (String[] args)
     {
        Student S = new Student ();
        S.display ();
     }
}
```

Key Points

- `final` class **cannot be inherited**.
- `final` class **cannot be extended** by another class.
- **Methods** inside a **final** class can be overridden only if the class could be sub classed—but since it cannot, **there can be no subclass overriding them**.
- `final` class can have constructors, variables, and methods normally.
- class cannot be both `abstract` and `final` because:
    - `abstract` requires inheritance.
    - `final` prevents inheritance.

Abstract Classes in Java:

An **abstract class** in Java is a class that is declared using the `abstract` keyword. It is used as a **base class** for other classes and **cannot be instantiated directly**.

```
abstract class Animal
{
    abstract void sound ();

    void eat ()
     {
        System.out.println ("Animal eats");
     }
}

class Dog extends Animal
{
     void sound ()
     {
        System.out.println ("Dog barks");
     }
}

public class Sample
{
    public static void main (String [] args)
    {
        Dog D = new Dog ();
        D.sound ();
        D.eat ();
    }
}
```

Abstract Method in Java:

An **abstract method** is a method that has **no body**. It is declared using the `abstract` keyword. **abstract method** must be **defined** in the **subclass.**

```
abstract void sound ();      // No Body
```

Key Points

- An abstract class **cannot be instantiated**.
- It can contain **abstract and non-abstract methods**.
- It can have **constructors**.
- It can contain **variables and static methods**.
- A subclass must implement all abstract methods, unless the subclass is also abstract.
- Abstract classes are useful for achieving **abstraction**.
- An abstract class can be used as a **reference type**.

Visibility Controls in Java:

Visibility controls in Java are also called **access modifiers**. They determine **where a class, variable, method, or constructor can be accessed**.

Java has **four levels of access controls**:

1. `public`
2. `protected`
3. `default` **(no keyword)**
4. `private`

1. `public` **member** can be accessed **from anywhere** in the **program**.
2. `protected` member can be accessed **within** the **same package**, In **subclasses**, including **subclasses** in **other packages**.
3. When no access modifier is specified, it is called **default access** or **package-private**. It can be accessed **only within the same package**.
4. `private` **member** can be accessed **only within the same class**.

```
class Student
{
    private int a = 10;
    int b = 20;              // default
    protected int c = 30;
    public int d = 40;
}
```

In the above example:

a → accessible only inside Student
b → accessible within the same package
c → accessible within the same package and through inheritance
d → accessible from anywhere

Access Modifier	Same Class	Same Package	Subclass in Different Package	Other Package
private	✓	✗	✗	✗
default	✓	✓	✗	✗
protected	✓	✓	✓	✗
public	✓	✓	✓	✓

Strings in Java:

String in Java is an **object used** to **store** a **sequence** of **characters** enclosed in **double** quotes. Such as **"Hello"** or **"Java Programming"**. **Strings** are one of the **most commonly used classes** in Java. An important property of Java strings is that they are **immutable**, meaning once a string object is created, its value cannot be changed. Any **operation** that appears to **modify** a **string** actually **creates** a **new string**.

Common String Methods in Java

Method	Description	Example
length()	Returns the length of the string	"Java".length() → 4
charAt(index)	Returns the character at the specified index	"Java".charAt(1) → 'a'
toUpperCase()	Converts to uppercase	"java".toUpperCase() → "JAVA"
toLowerCase()	Converts to lowercase	"JAVA".toLowerCase() → "java"
substring(start)	Returns part of the string	"Java".substring(2) → "va"
substring(start, end)	Returns a portion of the string	"Java".substring(1,3) → "av"
equals()	Compares string contents	"abc".equals("abc") → true
equalsIgnoreCase()	Compares ignoring case	"Java".equalsIgnoreCase("java") → true
contains()	Checks if a substring exists	"Java".contains("av") → true
replace()	Replaces characters or text	"Java".replace('a','o') → "Jovo"
trim()	Removes leading and trailing spaces	" Java ".trim() → "Java"

String Concatenation: Strings can be **joined** using the **+ operator**:

```
String Str1 = "Akshara";
String Str2 = "D G";
String Name = Str1 + " " + Str2;
System.out.println(Name);
```

Output: Akshara D G

String is Immutable: Once a `String` object is created, its value cannot be changed.

```
String S = "Java";
S.concat(" Programming");
System.out.println(s);
```

Output: Java

```
s = s.concat(" Programming");
System.out.println(s);
```

Output: Java Programming

```
import java.lang. String;
class StringDemo
{

  Public static void main(String args[])
   {

    String S1=new String ("c v raman college");
    String S2="C V RAMAN COLLEGE";
    System.out.println ("The string S1 is: " +S1);
    System.out.println(" The string S1 is: " +S2);
    System.out.println("Length of the string S1 is:"+S1.length());
    System.out.println("The first occurrence of r is at the position:"+S1.indexOf('r'));
    System.out.println("The String S1 in Upper Case: " +S1.toUpperCase());
    System.out.println("The String S1 in Lower Case: " +S1.toLowerCase());
    System.out.println("S1 Equals to S2: " +S1.equals(S2));
    System.out.println("S1 equals ignore case to S2: "+S1.equalsIgnoreCase(S2));
    int result=S1.compareTo(S2);
     System.out.println("After Compare To(): ");
    if(result == 0)
    System.out.println(S1+"is Equal to"+S2);
    else if(result>0)
    System.out.println(S1+" is Greater than to "+S2);
    else
    System.out.println( S1 + " is Smaller than to "+S2);
    System.out.println("Character at an index of 6 is: "+S1.charAt(6));
    String S3=S1.substring(5,9);
    System.out.println("Extracted Sub String is: "+S3);
    System.out.println("After Replacing c with d in S1: "+S1.replace('c','d'));
    String S4=" of administration and network sciences";
    System.out.println("The string S4 is: "+S4);
    System.out.println("After trim():"+S4.trim());
  }
}
```

Output:

```
The string S1 is:   c v raman college
The string S2 is:   C V RAMAN COLLEGE
Length of the string S1 is: 17
The first occurrence of r is at the position: 5
The String S1 in Upper Case: C V RAMAN COLLEGE
The String S1 in Lower Case: c v raman college
S1 Equals to S2: false
S1 equals ignore case to S2: true
After Compare To ():
c v raman college is Greater than C V RAMAN COLLEGE
Character at an index of 6 is: a
Extracted Sub String is: raman
After Replacing c with d in S1: d v raman dollege
The string S4 is: of administration and network sciences
After trim(): ofadministrationandnetworksciences
```

String Buffer Class in Java:

StringBuffer is **peer class** of **String**. While **String creates** a **string** of **fixed length**, **StringBuffer** creates **strings** of **flexible length** and that can be **modified** in terms of both **length** and **content**. We can **insert** characters and substring in the **middle** of the **string**, OR **append another string** at the end of a **string**.

```
import java.lang.String;
class StringBufferDemo
{

 public static void main(String args[])
  {

   StringBuffer SB=new StringBuffer("This is my college");
   System.out.println("This String SB is: " +SB);
   System.out.println("The Length of the String SB is:"+ SB.length());
   System.out.println("The capacity of the string SB is : " + SB.capacity());
   System.out.println("The Character at an index of 6 is:"+ SB.charAt(6));
   SB.setCharAt(3,'x');
   System.out.println("After Setting Char x at position 3: " + SB);
   System.out.println("After Appending:"+ SB.append("in Davanagere"));
   System.out.println("After Inserting: " +SB.insert(19,"C V Raman "));
   System.out.println("After Deleting: " + SB.delete(19,22));
  }

}
```

Output:

```
This String SB is: This is my college
The Length of the String SB is: 18
The capacity of the string SB is: 34
The Character at an index of 6 is: s
After Setting Char x at position 3: Thix is my college
After Appending: This is my college in Davanagere
After Inserting: This is my college C V Raman in Davanagere
After Deleting: This is my college in Davanagere
```

Vectors in Java:

Vector is **class** in **java** can be used to **create generic dynamic array**, which can holds **objects of any type** and of **any number**. The objects **do not** have to be **homogeneous**. **Vectors** can be **declared without specifying** any **size** explicitly, and vector can **accommodate** an **unknown number** of **items**.

Syntax:

```
Vector  V = new  Vector ();
```

```
import java.lang.*;
import java.util.Vector;
import java.util.Enumeration;
class VectorDemo
{
 public static void main(String arg[])
 {
  Vector V=new Vector();
  V.addElement("One");
  V.addElement("Two");
  V.addElement("Three");
  V.insertElementAt("Zero",0);
  V.insertElementAt("Oops",3);
  V.insertElementAt("Four",5);
  System.out.println("Vector Size:"+V.size());
  System.out.println("Vector apacity:"+V.capacity());
  System.out.println("The Elements ofa vector are:");
  Enumeration E=V.elements();
  while(E.hasMoreElements())
  System.out.println(E.nextElement() +"");    System.out.println();
  System.out.println("The first element is: " +V.firstElement());
  System.out.println("The last element is: " +V.lastElement());
  System.out.println("The object oops is found at position:"+V.indexOf("oops"));
  V.removeElement("oops");
  V.removeElementAt(1);
  System.out.println("After removing 2 elements");
  System.out.println("Vector Size:"+V.size());
  System.out.println("The Elements of vector are:");
  for(int i=0;i<V.size();i++)
  System.out.println(V.elementAt(i)+"");
 }
}
```

Output:

```
Vector Size: 6
Vector Capacity: 10
The Elements of a vector are:
Zero
One
Two
Oops
Three
Four
The first element is: Zero
The last element is: Four
The object oops is found at position: 3
After removing 2 elements
Vector Size: 4
The Elements of a vector are:
Zero
Two
Three
Four
```

Vector in **Java cannot store primitive data types directly** because **Vector stores Objects** Only. When we add a **primitive value** to a **Vector**, Java **automatically converts** it into its **corresponding wrapper class** using **Autoboxing**.

```
import java.util.Vector;
class VectorDemo
{
 public static void main(String args[])
 {
   Vector V<Integer> = new Vector<>();
   V.add(10);   V.add(20);   V.add(30);   int x = V.get(10);
   System.out.println(V);
   System.out.println(x);
 }
}
```

Output:

```
[10, 20, 30]
10
```

What happens internally…?

V.add(10); → is treated as V.add(Integer.valueOf(10));

Similarly,

int x = V.get(0); → is treated as int x = V.get(0).intValue();

Summary

- Vector stores Objects, not primitive data types.
- Primitive values (int, char, float, double, etc) are automatically converted wrapper objects (Integer, Character, Float, Double, etc) through Autoboxing.
- When retrieving values, Java automatically converts wrapper objects back to primitive values through Unboxing.

Wrapper Classes in Java:

Java Wrapper Classes are the class that **converts primitive data types** into **Objects**. They are the part of the **java**.**lang** package.

Primitive data types and their Wrapper Classes.

Primitive Type	Wrapper Class
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character
boolean	Boolean

```
class WrapperClass
{
 public static void main(String args[])
 {
     int a = 10;
     Integer Obj = a;
     int b = Obj;
     System.out.print("Primitive: "+a);
     System.out.print("Wrapper Object: "+Obj);
     System.out.print("Unboxed Primitive: "+b);
 }
}
```

Output: `Primitive: 10 Wrapper Object: 10 Unboxed Primitive: 10`

Autoboxing: Automatic conversion of primitive data type into its wrapper object.
`Integer x = 100;`

Unboxing: Automatic conversion of wrapper objects into its primitive data type.
`int y = x;`

Wrapper Classes allow **primitive values** to be treated as **objects**, making them useful with **collections**, **generics**, and **object oriented features** in **Java**.

Static Members in Java:

In Java, a **static data member** (also called a **static variable** or **class variable**) is a variable that belongs to the **class itself**, not to individual objects. This means there is only **one copy** of the variable, shared by all objects of the class.

```
class Student
{
    String name;
    static String college = "ABC College";
    Student (String n)
    {
        name = n;
    }
    void display()
    {
        System.out.println (name + " studies at " + college);
    }
}
public class Sample
{
    public static void main(String[] args)
    {
        Student s1 = new Student ("Gandharvaa");
        Student s2 = new Student ("Akshara");

        s1.display();
        s2.display();
    }
}
```

Output:
```
Gandharvaa studies at ABC College
Akshara studies at ABC College
```

Characteristics of Static Data Members:

- Belong to the **class**, not to objects.
- Memory is allocated **only once** when the class is loaded.
- Shared among all objects of the class.
- Can be accessed using the class name:

```
Student.college = "XYZ College";
```

Advantages:

- **Saves memory** because only **one copy exists**.
- Useful for values **common** to **all objects** (e.g., company name, college name, interest rate).
- Can be used to **keep track** of **shared information**, such as the **number** of **objects created**.

Static Variable	Instance Variable
Belongs to the Class	Belongs to an Object
One copy shared by all Objects	Separate copy for each Object
Accessed by using the class name	Accessed by using the Object
Memory is allocated once	Memory is allocated for each Object

Static Class in Java:

In Java, **static classes** are a special type of **nested class**. A **top-level class cannot be declared `static`**. Only a class defined inside another class can be static.

```
class Outer
{
    static int x = 100;
    static class Inner
    {
        void show()
        {
            System.out.println("Value of x = " + x);
        }    }   }
public class Sample
{
    public static void main(String[] args)
    {
        Outer.Inner obj = new Outer.Inner();
        obj.show();
    }
}
```

Output: `Value of x = 100`

Features of Static Nested Classes:

Declared with the `static` keyword inside another class.

Belong to the outer class, not to an instance of it.

Can access **only static members** of the outer class directly.

Cannot directly access non-static members.

Can have their own fields, methods, and constructors.